do a performant sort on one key, but also to define two kinds of ordering on the data: one that governs partitioning and another that governs the ordering of elements on the child partitions. The rest of this section will focus on the second use case, in which we want to organize the data first by one ordering and next by another.

Leveraging repartitionAndSortWithinPartitions for a Group by Key and Sort Values Function

The best way to order data with two orderings is to use the `repartitionAndSortWithinPartitions` function. One common use case for this is to a define a function, which we might call "group by key and sort values," that returns an RDD grouped by key with the values in each group sorted. Unlike sorting by tuple keys, which we discussed in "Sorting by Two Keys with SortByKey", this approach could be generalized to any partitioning defined on any key type and any custom ordering. We use `repartitionAndSortWithinPartitions` to repartition the RDD by one ordering on the keys and then define an implicit ordering for the records on a given partition.

With a little searching you can find numerous `groupByKeyAndSortValues` functions, although none of them have been merged into Spark. Sandy Ryza's presentation in *Advanced Analytics with Spark* is particularly good, and the following implementation closely mirrors his. This `groupByKeyAndSortValues` function assumes data in the form `((k, s), v)`, where `s` is the secondary key (perhaps derived from the value). It partitions the RDD by the first part of the key, then sorts by the second part of the key. Then, it then combines all the values associated with one key into a sorted iterator.

The function has four steps:

1. Define a custom partitioner that partitions records according to the first element of the key.
2. Define an implicit ordering on the values. This is only necessary because the function is generic. The implicit ordering on tuples is first value, second value. We just have to tell Spark to use that tuple ordering.
3. Use `repartitionAndSortWithinPartitions` on the input RDD with the custom partitioner defined in step 1.
4. Coalesce the items using a `mapPartitions` routine. We can leverage the fact that items with the same first key are on the same partition and that the elements within each partition are sorted first by the first ordering and then by the second ordering.

> **NOTE**
>
> Because we are using hash partitioning, this function does not actually sort values by the first key. Rather, it groups keys with the same hash value on the same machine. Thus, if we run the function of the values one through five and use four partitions, the first partition will contain one and five. To force the keys to appear in true sorted order, we would need to define a range partitioner. However, using the hash value is good enough if our goal is simply to group like keys.

Example 6-10 is the code for the custom partitioner. As you can see we order only on the first part of the key. If we define the ordering on both parts of the key, the partitioner will group by the hash value of the entire tuple key. Consequently the function may put elements with the same primary key, but different secondary keys on two different partitions.

Example 6-10. Step 1 of secondary sort: defining a custom partitioner

```
class PrimaryKeyPartitioner[K, S](partitions: Int) extends
```

```
Partitioner {
  /**
   * We create a hash partitioner and use it with the first set of
keys.
   */
  val delegatePartitioner = new HashPartitioner(partitions)

  override def numPartitions = delegatePartitioner.numPartitions

  /**
   * Partition according to the hash value of the first key
   */
  override def getPartition(key: Any): Int = {
    val k = key.asInstanceOf[(K, S)]
    delegatePartitioner.getPartition(k._1)
  }
}
```

Next, we define the implicit ordering, as shown in Example 6-11. Recall from the specification of the function, that both parts of the key already have an ordering defined. Spark already has an ordering for tuples of two elements with orderings so we can simply tell Spark to use the generic Tuple2 ordering.

Example 6-11. Step 2 of secondary sort: define the implicit ordering

```
    implicit def orderByLocationAndName[A <: PandaKey]:
Ordering[A] = {
      Ordering.by(pandaKey => (pandaKey.city, pandaKey.zip,
pandaKey.name))
    }

    implicit val ordering: Ordering[(K, S)] = Ordering.Tuple2
```

Now, incorporating these two subroutines, we can define a function `groupByKeyAndSortBySecondaryKey`, as in Example 6-12. The new function will partition according to the partitioner defined in step 1, sort by the order defined in step 2, and then use a `groupSorted` function to combine the elements with the same first key into a single iterator.

Example 6-12. A general example of a "group by key and sort by secondary key" function

```
  def groupByKeyAndSortBySecondaryKey[K : Ordering : ClassTag,
    S : Ordering : ClassTag,
    V : ClassTag]
    (pairRDD : RDD[((K, S), V)], partitions : Int):
      RDD[(K, List[(S, V)])] = {
    //Create an instance of our custom partitioner
    val colValuePartitioner = new PrimaryKeyPartitioner[Double,
Int](partitions)

    //define an implicit ordering, to order by the second key the
ordering will
    //be used even though not explicitly called
    implicit val ordering: Ordering[(K, S)] = Ordering.Tuple2

    //use repartitionAndSortWithinPartitions
    val sortedWithinParts =

pairRDD.repartitionAndSortWithinPartitions(colValuePartitioner)

    sortedWithinParts.mapPartitions( iter => groupSorted[K, S, V]
(iter) )
  }

  def groupSorted[K,S,V](
    it: Iterator[((K, S), V)]): Iterator[(K, List[(S, V)])] = {
    val res = List[(K, ArrayBuffer[(S, V)])]()
    it.foldLeft(res)((list, next) => list match {
      case Nil =>
        val ((firstKey, secondKey), value) = next
        List((firstKey, ArrayBuffer((secondKey, value))))

      case head :: rest =>
        val (curKey, valueBuf) = head
        val ((firstKey, secondKey), value) = next
        if (!firstKey.equals(curKey) ) {
          (firstKey, ArrayBuffer((secondKey, value))) :: list
        } else {
          valueBuf.append((secondKey, value))
          list
        }

    }).map { case (key, buf) => (key, buf.toList) }.iterator
  }
```

> **TIP**
>
> When developing a function like this one that relies heavily on partitioning, make sure that your unit tests are for data that spans more than one partition. Make sure to test on different numbers of partitions and different data, because there is some randomness in partitioning (especially range partitioning). See Chapter 8 for more about running good distributed tests.

How Not to Sort by Two Orderings

It's important to note that several other seemingly obvious approaches to this problem are not guaranteed to give the correct result. For example, even regardless of performance issues, using `groupBykey` does not maintain the order of the values within the groups. Thus the following implementation may not give the correct results:

```
indexValuePairs.sortByKey().groupByKey()
```

Spark sorting is also not guaranteed to be stable (preserve the original order of elements with the same value). Hence, repeated sorting is not a viable option:

```
indexValuePairs.sortByKey.map(_.swap()).sortByKey
```

In this case, the second `sortByKey` may not preserve the ordering generated in the first sort.

Goldilocks Version 2: Secondary Sort

The logic of secondary sort generalizes well beyond simply ordering data. It applies to any use case that requires the records to be arranged

according to two different keys. The original Goldilocks example is related to secondary sort since it requires us to shuffle on one key (the column index) and then order the data within each key by value (by the value in the cells). Thus, rather than using `groupByKey` to ensure that the values associated with each key are coalesced and then sorting the elements associated with each key as a separate step, we can use `repartitionAndSortWithinPartitions`. Using `repartitionAndSortWithinPartitions` we can partition on the column index and sort on the value in each column. We are then guaranteed that all the values associated with each column will be on one partition and that they will be in sorted order of the value. We can simply loop through the elements on each partition, filter for the desired rank statistics in one pass through the data, and use the `groupSorted` function to combine the rank statistics associated with our column.

DEFINING THE CUSTOM PARTITIONER

The ordering and partition in `repartitionAndSortWithinPartitions` must be defined on the keys of the RDD, and thus we need to use the (column index, value) pairs as keys. We can map to a dummy value (like 1 or null) so that Spark will interpret the RDD as key/value pairs where the keys are a tuple of `(column index, value)`. We will then need to define a custom partitioner that partitions the keys based on the hash value of the first part of the key (the column index), as shown in Example 6-13.

Example 6-13. Goldilocks version 2, defining a custom partitioner to partition on column index

```
class ColumnIndexPartition(override val numPartitions: Int)
  extends Partitioner {
  require(numPartitions >= 0, s"Number of partitions " +
    s"($numPartitions) cannot be negative.")
```

```
  override def getPartition(key: Any): Int = {
    val k = key.asInstanceOf[(Int, Double)]
    Math.abs(k._1) % numPartitions //hashcode of column index
  }
}
```

FILTERING ON EACH PARTITION

On each partition, we want the elements to be ordered first by column index, and second by value. The former ensures that all the records associated with one key are on the same partition. The latter ensures that the elements that are adjacent will be those with the same column indices and will be sorted so that we can find the rank statistics.

Ordering by the first value of a tuple and then the second is the existing implicit ordering on tuples. Thus, we do not have to specify an ordering for our data. After the `repartitionAndSortWithinPartitions` call, we know that the data will be partitioned according to column index and sorted by column index and value. For example, suppose that we were using the `DataFrame` described in Table 6-1 and that we were using three partitions. The first partition would contain the following values:

((1, 2.0), 1)
((1, 3.0), 1)
((1, 10.0), 1)
((1, 15.0), 1)
((4, 0.0), 1)
((4, 0.0), 1)
((4, 0.0), 1)
((4, 98.0), 1)

We can use the filter operation to loop through the elements of the iterator even though the filter requires us to keep track of global data. Recall from our discussion of iterator-to-iterator transformations in "Iterator-to-Iterator Transformations with mapPartitions" that the `map`, `filter`, and `flatMap` operations defined on iterators transform the elements in the iterator in order. Thus, because the elements are sorted and grouped by key, we can keep track of a running total for the column index. If the element is one of the ones that corresponds to the target ranks statistic, then we can keep it. We then have to map the iterator to the first half of the tuple to remove the `1` dummy value. Note that we could combine these `map` and `filter` steps into one `flatMap` operation. We have chosen to present them separately in Example 6-14 since we think that the `filter` operation is easier to interpret.

Example 6-14. Goldilocks version 2, leveraging repartitionAndSortWithinPartitions

```
  def findRankStatistics(dataFrame: DataFrame,
    targetRanks: List[Long], partitions: Int) = {

    val pairRDD: RDD[((Int, Double), Int)] =
      GoldilocksGroupByKey.mapToKeyValuePairs(dataFrame).map((_,
1))

    val partitioner = new ColumnIndexPartition(partitions)
     //sort by the existing implicit ordering on tuples first key,
second key
    val sorted =
pairRDD.repartitionAndSortWithinPartitions(partitioner)

    //filter for target ranks
    val filterForTargetIndex: RDD[(Int, Double)] =
      sorted.mapPartitions(iter => {
        var currentColumnIndex = -1
        var runningTotal = 0
        iter.filter({
          case (((colIndex, value), _)) =>
            if (colIndex != currentColumnIndex) {
              currentColumnIndex = colIndex //reset to the new
column index
```

```
              runningTotal = 1
            } else {
              runningTotal += 1
            }
          //if the running total corresponds to one of the rank
statistics.
          //keep this ((colIndex, value)) pair.
          targetRanks.contains(runningTotal)
      })
    }.map(_._1), preservesPartitioning = true)
    groupSorted(filterForTargetIndex.collect())
  }
```

COMBINE THE ELEMENTS ASSOCIATED WITH ONE KEY

After the `mapPartitions` step, we have to do one last local transformation to group the elements associated with one column index into a map. The code for the `groupSorted` function is presented in Example 6-15.

Example 6-15. Goldilocks version 2, group the elements associated with one key

```
  private def groupSorted(
    it: Array[(Int, Double)]): Map[Int, Iterable[Double]] = {
    val res = List[(Int, ArrayBuffer[Double])]()
    it.foldLeft(res)((list, next) => list match {
      case Nil =>
        val (firstKey, value) = next
        List((firstKey, ArrayBuffer(value)))
      case head :: rest =>
        val (curKey, valueBuf) = head
        val (firstKey, value) = next
        if (!firstKey.equals(curKey)) {
          (firstKey, ArrayBuffer(value)) :: list
        } else {
          valueBuf.append(value)
          list
        }
    }).map { case (key, buf) => (key, buf.toIterable) }.toMap
  }
```

Notice that this code is very similar to the grouping function presented in

Example 6-12.

PERFORMANCE

This solution is considerably faster than the version 1 `groupByKey` solution on any shape of data. By using `repartitionAndSortWithinPartitions`, we are able to push the work to sort each column into the shuffle stage. Since the elements are sorted after the shuffle, we are able to use iterator-to-iterator transformations to filter the data and avoid forcing all the values associated with one partition into memory.

However, if the columns are relatively long, the `repartitionAndSortWithinPartitions` step may still lead to failures since it still requires one executor to be able to store all of the values associated with all of the columns that have the same hash value. Indeed, in our case we still saw failures in the shuffle stage using this approach at scale. In fact, a viable solution to the Goldilocks problem required taking an entirely different approach.

A Different Approach to Goldilocks

Unfortunately, none of the existing key/value transformations provided a magic bullet for the Goldilocks problem. None of the other aggregation operations that we might use as alternative to `groupByKey` help us since the operation that we want to perform for each key—a sort—won't reduce the size of the data by key. As we discussed in the previous section, even rewriting our `groupByKey` approach using sophisticated secondary sort techniques was leading to failures. In the end, the secondary sort approach still required partitioning by the column index, which was not granular enough for the size of our data and the resources we had available.

Instead, a performant solution to this problem required entirely rethinking how we parallelized the computational work.

Before we dive into the solution, let's review some of the methods we have learned to make transformations more performant:

- Narrow transformations on key/value data are quick and easy to parallelize relative to wide transformations that cause a shuffle.
- Partition locality can be retained across some narrow transformations following a shuffle. This applies to `mapPartitions` if we use `preservePartitioning=true`, or `mapValues`.
- Wide transformations are best with many unique keys. This prevents shuffles from directing a large proportion of the data to reside on one executor.
- `SortByKey` is a particularly good way to partition data and sort within partitions since it pushes the ordering of data on local machines into the shuffle stage.
- Using iterator-to-iterator transforms in `mapPartitions` prevents whole partitions from being loaded into memory.
- We can sometimes rely on shuffle files to prevent recomputation of wide transformations even if we call several actions on the same RDD.

Using these insights, we were able to construct a solution to the Goldilocks problem using only one `sortByKey` and three `mapPartitions` operations. The critical insight is that the unit of parallelization for this problem does not need to be the columns. We can essentially solve the problem for each range of values. If the cell values are sorted and we know how many elements are on each partition from each column (which is easy to calculate using a performant `mapPartitions` routine), then we can determine the location of the

`nth` element.

Our solution can be enumerated in five steps:

1. Map the rows of data to pairs of `(cell value, index)`.
2. Perform a `sortByKey` operation on all tuples defined in step 1.
3. Using `mapPartitions`, determine how many elements in each column are on each partition and collect that information to the driver.
4. Perform a local computation on the result of step 3 to determine the location of each desired rank statistic. For example, suppose that we are looking for the 13th element. Suppose also that in step 3 we determined that the first partition had 10 elements from column six. In this case, we can conclude that the 13th element will be the third largest element in column six on the second partition.
5. Using the result of step 4, use another `mapPartitions` transformation to filter for the elements that correspond to the desired rank statistics. Collect this information back to the driver.

MAP TO (CELL VALUE, COLUMN INDEX) PAIRS

Example 6-16 is the code for step 1 of our solution: mapping to the `(cell value, column index)` pairs. We use Spark's `flatMap` function to transform each row into a sequence of tuples.

Example 6-16. Goldilocks algorithm version 3, map to (cell value, column index) pairs

```
  private def getValueColumnPairs(dataFrame : DataFrame):
RDD[(Double, Int)] = {
    dataFrame.rdd.flatMap{
      row: Row => row.toSeq.zipWithIndex
                  .map{
                    case (v, index) => (v.toString.toDouble,
index)}
    }
  }
```